

中国科学技术大学

密码学导论课程作品报告

基于 AES-256-GCM 的加密文件库系统

—— 可否认加密机制的设计与实现

姓名	谢家兴
学号	PB24061264
课程名称	密码学导论
实验主题	可否认加密文件库
完成日期	2026 年 6 月
技术栈	Python 3.10+ / AES-256-GCM / PBKDF2 / HKDF

目录

摘 要.....4

1 引言4

 1.1 背景.....4

 1.2 设计目标.....4

2 加密原理.....5

 2.1 主密钥派生：PBKDF2-HMAC-SHA256.....5

 2.2 每文件密钥派生：HKDF-SHA2565

 2.3 数据加密：AES-256-GCM.....5

 2.4 密码验证机制5

 2.5 欺骗数据生成6

3 可否认性实现思路6

 3.1 双密钥体系6

 3.2 密钥路由机制6

 3.3 不可区分性保证.....6

 3.4 攻击模型分析7

4 系统架构.....7

 4.1 分层架构图7

 4.2 存储布局.....7

 4.3 数据模型.....8

 4.4 加密流程.....8

 4.5 解密流程.....8

5 关键代码分析9

 5.1 加密函数（core/crypto.py）9

5.2	密钥路由 (storage/vault.py)	9
5.3	欺骗数据生成 (core/crypto.py)	9
6	测试与验证	10
6.1	测试覆盖	10
6.2	典型测试场景结果	10
7	总结与展望	10
7.1	已实现功能	10
7.2	未来改进方向	11
	参考文献	11
	附录 1: CLI 命令速查表	12
	附录 2: github 仓库地址	12

摘要

本文设计并实现了一个基于 AES-256-GCM 认证加密与 PBKDF2-HMAC-SHA256 密钥派生函数的本地加密文件库系统。该系统支持标准的文件加密存储、解密导出、增删查操作，并创新性地引入可否认加密（Plausible Deniability）机制：用户可以设置两套独立密码——真实密码解出原始文件，备用欺骗密码解出合法外观的善本文档——攻击者在仅获得密文和元数据的情况下无法区分两者的存在。系统采用分层密钥架构（PBKDF2 → 主密钥 → HKDF → 每文件子密钥），确保了密钥泄露的横向隔离。

关键词：AES-256-GCM, PBKDF2, HKDF, 可否认加密, 认证加密, 密钥派生

1 引言

1.1 背景

在数字化时代，个人和组织面临日益严峻的数据泄露和强制解密威胁。传统的加密系统虽然能保护数据机密性，但加密文件的存在本身就是一种信号——攻击者（或胁迫者）可据此要求用户交出密钥。可否认加密（Plausible Deniability）通过隐藏“有秘密”这一事实，为用户提供更深层的安全保障。

1.2 设计目标

1. 数据机密性：加密文件对未授权方不可读
2. 数据完整性：检测任何密文篡改
3. 可否认性：对同一密文库，不同密码解密出不同但均合法的明文
4. 不可区分性：攻击者检查文件库元数据时，无法判断是否存在“真正”的秘密文件
5. 密钥隔离：单文件密钥泄露不影响其他文件

2 加密原理

2.1 主密钥派生：PBKDF2-HMAC-SHA256

系统使用 PBKDF2 (Password-Based Key Derivation Function 2, RFC 2898) 从用户密码派生出 256 位主密钥。

```
master_key = PBKDF2-HMAC-SHA256(password, salt, iterations=600,000, dkLen=32)
```

参数选择依据如下表所示：

参数	取值	依据
哈希函数	SHA-256	FIPS 180-4 标准
迭代次数	600,000	OWASP 2023 推荐下限
盐长度	128 bit (16 B)	NIST SP 800-132 建议 $\geq$ 128 bit
输出长度	256 bit (32 B)	匹配 AES-256 密钥长度

2.2 每文件密钥派生：HKDF-SHA256

为防止单文件密钥泄露导致其他文件连带暴露，系统使用 HKDF (HMAC-based Key Derivation Function, RFC 5869) 为每个文件派生独立的 AES 密钥。

```
file_key = HKDF-SHA256(IKM=master_key, salt=per_file_salt, info="vault-file-key-v1", L=32)
```

安全性分析：HKDF 的 Extract-then-Expand 结构确保：即使攻击者获得某个 file_key，无法逆推 master_key (HKDF 的单向性)；不同 per_file_salt 产生独立、不可区分的 file_key (伪随机性)。

2.3 数据加密：AES-256-GCM

系统采用 AES-256-GCM (Galois/Counter Mode, NIST SP 800-38D) 进行认证加密。GCM 模式的优势在于：一次性提供机密性和完整性，无需额外 HMAC；每次加密生成独立随机 nonce (96 bit)，冲突概率小于 2^{-32} ；Tag 长度为 128 bit (16 B)，提供强认证强度。

2.4 密码验证机制

系统初始化时，将固定魔数 "VAULT_OK" (8 字节) 使用主密钥加密并存入 master.check。后续任何操作前，先尝试解密该文件：解密成功且明文为 "VAULT_OK" 则密码正确；解密失败 (InvalidTag) 则密码错误。在可否认模式

下，系统同时保存两份验证文件：`master.check`（真实密钥）和 `master.decoy`（欺骗密钥）。

2.5 欺骗数据生成

欺骗数据通过 `gen_decoy_data()` 函数生成，核心约束为 `len(decoy_data) == len(real_data)` 严格保证，使得攻击者无法通过文件大小区分真实与欺骗。

文件类型	后缀	欺骗策略
文本文件	.txt .md .log .csv	循环填充正常中文善本文本
二进制文件	其余所有后缀	全部填充空格字节 (0x20)

3 可否认性实现思路

3.1 双密钥体系

可否认模式初始化时，用户设置两套独立密码：真实密码通过 PBKDF2 派生出 `real_key`，用于解密原始文件；欺骗密码通过 PBKDF2 派生出 `decoy_key`，用于解密善本文件。两者共享同一 `master.salt`，攻击者无法推断密码数量。

3.2 密钥路由机制

`vault_open()` 的密钥路由自动完成，外部调用者零感知。系统首先尝试用输入密码解密 `master.check`，若成功则判定为真实密钥；否则尝试解密 `master.decoy`，若成功则判定为欺骗密钥；两者均失败则抛出 `VaultPasswordError`。

3.3 不可区分性保证

系统在多个观察维度上保证真实与欺骗的不可区分性，如下表所示：

观察维度	真实	欺骗	可区分？
<code>vault list</code> 输出	相同	相同	不可区分
<code>manifest.json</code> 中 <code>size</code> 字段	相同	相同	不可区分
<code>master.salt</code>	共享	共享	不可区分
<code>master.check</code> / <code>master.decoy</code>	均存在	均存在	不可区分
<code>objects/*.enc</code> / <code>/*.decoy</code>	均存在	均存在	不可区分

密文长度	len(plaintext)	len(plaintext) (相同)	不可区分
------	----------------	---------------------	------

3.4 攻击模型分析

场景 A：攻击者获得 `.vault/` 目录的全部内容。攻击者看到两个验证文件和两份密文，无法判断哪份是“真实”。如果用户交出欺骗密码，攻击者解出的是合法善本文件，无法证明还存在“真实”数据。场景 B：攻击者比较 `vault list` 在不同密码下的输出，输出完全一致（`name`、`size`、`added` 字段），无法区分。

局限：如果攻击者具备密文长度 + 文件类型的外部知识（例如知道某 PDF 不可能只有 100 字节），可能怀疑欺骗数据。这是当前实现的已知限制，可通过填充到固定块大小来缓解（未在本项目中实现）。

4 系统架构

4.1 分层架构图

系统采用四层架构设计，从底层到顶层依次为：底层密码库（`cryptography/PyCA`）、密码学原语层（`core/crypto.py`）、业务逻辑层（`storage/vault.py`）和用户交互层（`cli/main.py`）。各层职责清晰，便于维护和扩展。

层级	模块	职责
用户交互层	<code>cli/main.py</code>	<code>argparse</code> + <code>getpass</code> ，提供 <code>init/add/remove/list/open</code> 五个子命令
业务逻辑层	<code>storage/vault.py</code>	密钥路由、元数据管理、 <code>VaultEntry</code> 数据模型
密码学原语层	<code>core/crypto.py</code>	<code>PBKDF2</code> + <code>HKDF</code> 、 <code>AES-256-GCM</code> 、善本数据生成、随机数生成
底层密码库	<code>cryptography (PyCA)</code>	<code>CFFI</code> → <code>OpenSSL</code> ，提供 <code>PBKDF2HMAC</code> / <code>HKDF</code> / <code>AESGCM</code>

4.2 存储布局

加密文件库的存储布局如下，所有数据存储在 `.vault/` 目录中：

存储布局结构

.vault/		
└─ master.salt	# 16 B	- PBKDF2 盐值（共享）
└─ master.check	# 36 B	- 魔数密文（真实密钥验证）
└─ master.decoy	# 36 B	- 魔数密文（欺骗密钥验证，可选）
└─ manifest.json	#	元数据列表 [VaultEntry, ...]
└─ objects/		
└─ a1b2c3.enc	#	真实密文 blob
└─ a1b2c3.decoy	#	欺骗密文 blob
└─ d4e5f6.enc	#	...
└─ d4e5f6.decoy	#	...

4.3 数据模型

VaultEntry 数据类包含公开字段（无密码可见）和加密参数字段。公开字段包括 name（逻辑文件名）、original_path（原始路径）、file_suffix（文件后缀）、size（原始明文大小）和 added（添加时间）。加密参数分为真实和欺骗两组，各包含 salt、nonce、tag 和 object_id。

4.4 加密流程

加密流程包括以下步骤：读取明文文件内容；认证密码（通过 PBKDF2 派生密钥并验证 master.check）；生成欺骗数据（gen_decoy_data 保证长度严格一致）；使用 HKDF 派生真实文件密钥并用 AES-GCM 加密真实数据；使用 HKDF 派生欺骗文件密钥并用 AES-GCM 加密欺骗数据；分别写入 objects/ 目录；更新 manifest.json。

4.5 解密流程

解密流程包括以下步骤：通过 PBKDF2 派生密钥；尝试解密 master.check，若匹配则角色为 real；否则尝试解密 master.decoy，若匹配则角色为 decoy；根据角色选择对应的加密参数；读取密文 blob；通过 HKDF 派生文件密钥；使用 AES-GCM 解密并写入输出文件。

5 关键代码分析

5.1 加密函数（core/crypto.py）

加密函数负责生成随机 nonce、执行 AES-256-GCM 加密，并分离密文和认证标签。

代码 5.1: AES-256-GCM 加密函数

```
def encrypt(plaintext: bytes, key: bytes, associated_data: bytes = b'') -> tuple:
    nonce = generate_nonce()          # 96-bit 随机 nonce
    aesgcm = AESGCM(key)
    ct_with_tag = aesgcm.encrypt(nonce, plaintext, associated_data)
    ciphertext = ct_with_tag[:-16]    # 分离密文和标签
    tag = ct_with_tag[-16:]
    return nonce, ciphertext, tag
```

5.2 密钥路由（storage/vault.py）

密钥路由函数自动判断输入密码对应真实密钥还是欺骗密钥，实现零感知切换。

代码 5.2: 密钥路由与认证

```
def _authenticate(password: str, root: Path) -> Tuple[bytes, str]:
    salt = (root / MASTER_SALT_FILE).read_bytes()
    key = derive_key(password, salt)
    if _try_check(key, root / MASTER_CHECK_FILE):    # 先试真实
        return key, "real"
    if _try_check(key, root / MASTER_DECOY_FILE):    # 再试欺骗
        return key, "decoy"
    raise VaultPasswordError("Incorrect master password.")
```

5.3 欺骗数据生成（core/crypto.py）

欺骗数据生成函数根据文件类型选择不同的填充策略，确保欺骗数据与真实数据长度完全一致。

代码 5.3: 欺骗数据生成函数

```
def gen_decoy_data(real_data: bytes, file_suffix: str) -> bytes:
    data_len = len(real_data)
    if file_suffix.lower() in frozenset({".txt", ".md", ".log", ".csv"}):
        # 循环填充中文善本
        buf = bytearray(data_len)
        template = _DECOY_TEMPLATE    # 中文 UTF-8 文本
        for i in range(data_len // len(template)):
```

```
        buf[i*len(template):(i+1)*len(template)] = template
        buf[-(data_len % len(template)):] = template[:data_len % len(template)]
        return bytes(buf)
    else:
        return b"\x20" * data_len # 二进制：全空格
```

6 测试与验证

6.1 测试覆盖

系统包含三个测试文件，共计 68 项自动化测试，覆盖密码学原语、可否认加密集成和端到端冒烟测试。

测试文件	测试数	覆盖范围	状态
test_crypto.py	45	PBKDF2 (7), HKDF (3), 随机生成器 (4), AES-GCM (14), 欺骗数据 (14), 工作流 (3)	通过
test_deniable.py	16	双密钥初始化、真实/欺骗加密、列表一致性、不可区分性	通过
smoke_test.py	7	端到端冒烟测试（单密钥模式）	通过
合计	68	密码学原语 + 集成测试 + 冒烟测试	全部通过

6.2 典型测试场景结果

test_crypto.py: 45 passed, 0 failed. test_deniable.py: 16 passed, 0 failed, 涵盖 vault init --decoy、vault add 真实与欺骗密文不一致、混合类型文件双加密、vault list 一致性、真实密钥解密、欺骗密钥解密、错误密码拒绝、双密钥删除、向后兼容、manifest size 相等性等场景。smoke_test.py: ALL SMOKE TESTS PASSED。

7 总结与展望

7.1 已实现功能

1. AES-256-GCM 认证加密/解密

2. PBKDF2-HMAC-SHA256 安全密钥派生 (600,000 迭代)
3. HKDF-SHA256 每文件密钥隔离
4. 可否认加密——双密钥、双密文、密钥自动路由
5. 完整 CLI 工具 (init / add / remove / list / open)
6. 非交互模式 (-k 参数)
7. 友好错误处理 (密码错误输出说明而非崩溃)
8. 68 项自动化测试

7.2 未来改进方向

1. 填充到固定块大小: 消除密文长度泄露的文件类型信息
2. 多层可否认: 支持嵌套可否认文件库 (Stenography-like)
3. 密钥文件支持: 除密码外, 支持使用密钥文件 (keyfile) 作为第二因素
4. Argon2id 迁移: 从 PBKDF2 迁移到更抗 GPU/ASIC 的 Argon2id
5. GUI 界面: 为降低非技术用户使用门槛, 开发图形界面

参考文献

- [1] NIST SP 800-38D — Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)
- [2] NIST SP 800-132 — Recommendation for Password-Based Key Derivation
- [3] RFC 2898 — PKCS #5: Password-Based Cryptography Specification v2.0
- [4] RFC 5869 — HMAC-based Extract-and-Expand Key Derivation Function (HKDF)
- [5] OWASP Password Storage Cheat Sheet (2023)
- [6] Canetti, R., et al. "Deniable Encryption." IACR Crypto 1997.

附录 1：CLI 命令速查表

以下为系统提供的全部 CLI 命令及其用法，便于快速查阅。

命令	说明	示例
<code>vault init</code>	初始化文件库	<code>vault init [-k password] [--decoy]</code>
<code>vault add</code>	加密文件入库	<code>vault add file.txt -k password [-n alias]</code>
<code>vault remove</code>	删除库内文件	<code>vault remove alias -k password</code>
<code>vault list</code>	列出所有文件（无需密码）	<code>vault list</code>
<code>vault open</code>	解密导出文件	<code>vault open alias -o out.txt -k password</code>

附录 2：github 仓库地址

仓库地址：https://github.com/Jason-Xie01/crypto_report.git